COMPTE RENDU TP2

Ex 1.1 Charger le fichier avec `brut = sc.textFile(DATA + "/ventes.csv", 3)`. Retirer l'en-tête : récupérer `entete = brut.first()` puis filtrer les lignes différentes de `entete` pour obtenir `donnees`.

```
1 brut = sc.textFile(DATA + "/ventes.csv", 3)
```

```
1 entete = brut.first()
```

```
1 donnees = brut.filter(lambda l: l != entete)
```

On a d'abord charger le fichier avec le brut, ensuite récupérer l'entête avec .first(), puis filtrer avec .filter()

Ex 1.2 Écrire une fonction `parse(ligne)` qui découpe la ligne sur la virgule et renvoie une **liste** contenant un tuple `(id, date, ville, produit, quantite, prix)` (types `int`, `str`, `str`, `str`, `int`, `float`) si la ligne est valide, et une **liste vide** sinon (utiliser `try / except (ValueError, IndexError)`).

[14]:
```
1 def parse(ligne):
2     try:
3         p = ligne.split(",")
4         return [(int(p[0]), p[1], p[2], p[3], int(p[4]), float(p[5]))]
5     except (ValueError, IndexError):
6         return []
```

Une fonction parse (ligne) qui a pour personnalité le découpage de la ligne avec .split() et (« , ») comme étant le paramètre pour la convertir en tuples avec des types bien definis.

Ex 1.3 Construire `ventes = donnees.flatMap(parse).cache()`.

1. Combien de lignes valides ? Combien de lignes rejetées ?
2. Pourquoi `flatMap` avec une liste de 0 ou 1 élément est-il plus pratique que `map` + `filter` ici ?
3. Que fait `cache()` ? Vérifier dans l'onglet **Storage** de l'interface 4040 après un premier `count()`.

[17]:
```
1 ventes = donnees.flatMap(parse).cache()
2
3 valides = ventes.count()
4 total = donnees.count()
5 print("valides :", valides, "| rejetées :", total - valides)
```

```
valides : 19 | rejetées : 3
```

Verifications de lignes valides et le total en faisant le compte avec .count()
FlatMap permet de transformer les lignes valides en tuples et éliminer les lignes invalides, resultat en cache . Cache stocke le resultat en mémoire RAM pour eviter de recalculer lors des actions qui vont suivre

Construire `ca = ventes.map(lambda v: (v[2], v[4] * v[5]))` : couples (ville, montant de la vente).

Ex 2.1 Chiffre d'affaires par ville avec `reduceByKey`.

```
[23]: 1 ca = ventes.map(lambda v: (v[2], v[4] * v[5]))

[26]: 1 ca_par_villes_reduce = ca.reduceByKey(lambda x, y : x + y)

[28]: 1 print("ca par ville (reduceByKey) :")

ca par ville (reduceByKey) :

[31]: 1 print(ca_par_villes_reduce.collect())

[('Casablanca', 34000.0), ('Fes', 35050.0), ('Rabat', 42100.0), ('Marrakech', 48600.0), ('Ag
adir', 14000.0)]
```

V(2)- représente les villes

V(4) et V(5)- représente la quantité et le prix
reduceByKey regroupe les elements entre eux et combine les valeurs selon les parametres
L'option collect ramène les resultats et ajoute les résultats sous forme de tuples contenant la ville et le chiffre d'affaire total.

Ex 2.2 Obtenir le même résultat avec `groupByKey().mapValues(sum)`. Comparer les deux plans avec `toDebugString()` et le volume de **Shuffle Write** de chaque job dans l'onglet **Stages**. Quelle méthode le cours recommande-t-il et pourquoi ?

```
[32]: 1 ca_par_ville_group = ca.groupByKey().mapValues(sum)

[33]: 1 print(ca_par_ville_group.collect())

[('Casablanca', 34000.0), ('Fes', 35050.0), ('Rabat', 42100.0), ('Marrakech', 48600.0), ('Ag
adir', 14000.0)]
```

Ex 2.3 Nombre de ventes par ville avec `countByKey()`. Quel est le type du résultat (RDD ou structure locale) ? Quel danger si le nombre de clés est très grand ?

```
[19]: nv = ventes.map(lambda v: (v[2], v[4] * v[5]))

[24]: nv_par_villes = nv.countByKey()

[27]: print(nv.countByKey())

defaultdict(<class 'int'>, {'Casablanca': 4, 'Rabat': 5, 'Fes': 4, 'Marrakech': 3, 'Agadir': 3})
```

CountByKey permet de regrouper les elements par clés faire le compte selon la clé choisie ici on a choisit les villes V(2) . Le danger serait une saturation de la mémoire donc surement un blocage

Ex 2.4 Quantité maximale vendue en une seule ligne, par produit, avec `aggregateByKey(0, max, max)`. Expliquer le rôle des trois arguments (valeur neutre, fonction intra-partition, fonction inter-partitions).

```
[21]: qmax = ventes.map(lambda v: (v[3], v[4])).aggregateByKey(0, max, max)
print(qmax.collect())
```

```
[('Tablette', 6), ('Clavier', 12), ('Laptop', 2), ('Telephone', 5), ('Souris', 30)]
```

Ce qu'il faut savoir c'est que le premier max compare les valeurs dans une même partition

Et le deuxieme max compare les résultats entre partitions

Ex 2.5 Convertir les montants en euros (taux 0,092) **sans toucher aux clés** avec `mapValues`. Vérifier que le nombre d'éléments et le nombre de partitions sont inchangés. Pourquoi `mapValues` est-il préférable à `map` ici ?

```
[50]: euros = ca_ville.mapValues(lambda m: m * 0.092)
print(euros.collect())
print(ca_ville.count(), euros.count())
print(ca_ville.getNumPartitions(), euros.getNumPartitions())
```

```
[('Casablanca', 3128.0), ('Fes', 3224.6), ('Rabat', 3873.2), ('Marrakech', 4471.2), ('Agadir', 1288.0)]
5 5
3 3
```

Mapvalue est préférable à map parce qu'il ne touche pas aux clés, donc il garde le partitionnement

Ex 2.6 Trier les villes par chiffre d'affaires décroissant de deux façons : avec `sortBy`, puis avec `sortByKey` (indice : inverser le couple, trier, ré-inverser).

```
[51]: # Avec sortBy
t1 = ca_ville.sortBy(lambda x: x[1], ascending=False)
print(t1.collect())

# Avec sortByKey : inverser, trier, ré-inverser
t2 = (ca_ville.map(lambda x: (x[1], x[0]))
              .sortByKey(ascending=False)
              .map(lambda x: (x[1], x[0])))
print(t2.collect())
```

```
[('Marrakech', 48600.0), ('Rabat', 42100.0), ('Fes', 35050.0), ('Casablanca', 34000.0), ('Agadir', 14000.0)]
[('Marrakech', 48600.0), ('Rabat', 42100.0), ('Fes', 35050.0), ('Casablanca', 34000.0), ('Agadir', 14000.0)]
```

Ex 2.7 Panier moyen par ville (CA / nombre de ventes) en **une seule** agrégation. Indice : valeur = `(montant, 1)`, `reduceByKey` qui additionne les deux composantes, puis `mapValues`.

[52]:
```
panier = (ventes.map(lambda v: (v[2], (v[4] * v[5], 1)))
              .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
              .mapValues(lambda p: p[0] / p[1]))
print(panier.collect())
```

```
[('Casablanca', 8500.0), ('Fes', 8762.5), ('Rabat', 8420.0), ('Marrakech', 1620
0.0), ('Agadir', 4666.666666666667)]
```

Partie 3 : Jointures

```
regions = sc.parallelize([
    ("Casablanca", "Casablanca-Settat"), ("Rabat", "Rabat-Sale-Kenitra"),
    ("Fes", "Fes-Meknes"), ("Tanger", "Tanger-Tetouan")], 2)
```

On note `ca_ville` le RDD du chiffre d'affaires par ville (Ex 2.1).

Ex 3.1 `ca_ville.join(regions)` : quelles villes disparaissent du résultat ? Pourquoi ?

[53]:
```
regions = sc.parallelize([
    ("Casablanca", "Casablanca-Settat"), ("Rabat", "Rabat-Sale-Kenitra"),
    ("Fes", "Fes-Meknes"), ("Tanger", "Tanger-Tetouan")], 2)
```

[54]:
```
print(ca_ville.join(regions).collect())
```

```
[('Casablanca', (34000.0, 'Casablanca-Settat')), ('Fes', (35050.0, 'Fes-Mekne
s')), ('Rabat', (42100.0, 'Rabat-Sale-Kenitra'))]
```

Il y a des villes qui disparaissent (Marrakech et agadir) car elles n'ont pas de region…

3.2

[55]:
```
print(ca_ville.leftOuterJoin(regions).collect())
print(ca_ville.rightOuterJoin(regions).collect())
```

```
[('Casablanca', (34000.0, 'Casablanca-Settat')), ('Fes', (35050.0, 'Fes-Mekne
s')), ('Rabat', (42100.0, 'Rabat-Sale-Kenitra')), ('Agadir', (14000.0, None)),
('Marrakech', (48600.0, None))]
[('Casablanca', (34000.0, 'Casablanca-Settat')), ('Fes', (35050.0, 'Fes-Mekne
s')), ('Rabat', (42100.0, 'Rabat-Sale-Kenitra')), ('Tanger', (None, 'Tanger-Teto
uan'))]
```

Ex 3.2 Comparer `leftOuterJoin` et `rightOuterJoin` avec `join`. Où voit-on apparaître `None` ?

NONE apparait pour Marrakech , agadir , tanger

[56]:
```
cg = (ca_ville.cogroup(regions)
              .mapValues(lambda p: (list(p[0]), list(p[1]))))
print(cg.collect())

cles = ca_ville.keys().union(regions.keys()).distinct().count()
print(cg.count(), cles)
```

```
[('Casablanca', ([34000.0], ['Casablanca-Settat'])), ('Fes', ([35050.0], ['Fes-M
eknes'])), ('Rabat', ([42100.0], ['Rabat-Sale-Kenitra'])), ('Agadir', ([14000.
0], [])), ('Tanger', ([], ['Tanger-Tetouan'])), ('Marrakech', ([48600.0], []))]
6 6
```

Partie 4 : Opérations ensemblistes et combinaisons

Ex 4.1 Soit `pc` les produits **distincts** vendus à Casablanca et `pr` ceux vendus à Rabat. Calculer `union` (sans doublons), `intersection` et `subtract` (produits vendus à Casablanca mais pas à Rabat).

[58]:
```
print("partie 4.1")
```

```
partie 4.1
```

[59]:
```
pc = ventes.filter(lambda v: v[2] == "Casablanca").map(lambda v: v[3]).distinct(
pr = ventes.filter(lambda v: v[2] == "Rabat").map(lambda v: v[3]).distinct()

print(pc.union(pr).distinct().collect())   # 5 produits
print(pc.intersection(pr).collect())       # Laptop, Telephone, Souris
print(pc.subtract(pr).collect())           # Clavier
```

```
['Tablette', 'Telephone', 'Clavier', 'Laptop', 'Souris']
['Telephone', 'Laptop', 'Souris']
['Clavier']
```

Ex 4.2 Calculer `villes.cartesian(produits)` avec les villes et produits distincts. Vérifier que `count == nb_villes * nb_produits`. Dans quel cas ce produit devient-il dangereux ?

[60]:
```
print("reponse 4.2")
```

```
reponse 4.2
```

[61]:
```
villes = ventes.map(lambda v: v[2]).distinct()
produits = ventes.map(lambda v: v[3]).distinct()
cart = villes.cartesian(produits)
print(cart.count(), villes.count() * produits.count())   # 25 et 25
```

```
25 25
```

Ex 4.3 Avec `a = sc.parallelize(range(1, 6), 2)` et `b = sc.parallelize(range(11, 16), 2)` : calculer `a.zip(b)`. Provoquer ensuite les **deux erreurs** classiques : (i) `b` avec 3 partitions ; (ii) `b` avec 6 éléments répartis sur 2 partitions. Quelle alternative utiliser (indice : `zipWithIndex`) ?

[63]:
```
a = sc.parallelize(range(1, 6), 2)
b = sc.parallelize(range(11, 16), 2)
print(a.zip(b).collect())   # (1,11) (2,12) (3,13) (4,14) (5,15)

# Erreur (i) : b avec 3 partitions
try:
    a.zip(sc.parallelize(range(11, 16), 3)).collect()
except Exception as e:
    print("Erreur 1 :", e)

# Erreur (ii) : 6 éléments sur 2 partitions
try:
    a.zip(sc.parallelize(range(11, 17), 2)).collect()
except Exception as e:
    print("Erreur 2 :", e)

# Alternative : zipWithIndex + join
ai = a.zipWithIndex().map(lambda x: (x[1], x[0]))
bi = sc.parallelize(range(11, 17), 2).zipWithIndex().map(lambda x: (x[1], x[0]))
print(ai.join(bi).values().collect())
```

```
[(1, 11), (2, 12), (3, 13), (4, 14), (5, 15)]
Erreur 1 : Can only zip with RDD which has the same number of partitions
Erreur 2 : Can not deserialize PairRDD with different number of items in batche
s: (2, 3)
[(1, 11), (5, 15), (2, 12), (3, 13), (4, 14)]
```

ZIP exige le même nombre de partitions et le même nombre d'éléments par partition.

ZIPwithIndex donne un numéro à chaque élément puis join les associe.

Partie 5 : Partitionnement et sauvegarde

Ex 5.1 Afficher `ventes.getNumPartitions()`. Appliquer `coalesce(1)`, `repartition(6)` et `coalesce(10)`. Que constatez-vous pour le dernier ?

[65]:
```
print(ventes.getNumPartitions())
print(ventes.coalesce(1).getNumPartitions())
print(ventes.repartition(6).getNumPartitions())
print(ventes.coalesce(10).getNumPartitions())
```

```
3
1
6
3
```

Coaleste ne change rien sans shuffle il diminue juste les partitions

Ex 5.3 Sauvegarder `ca_ville` avec `coalesce(1).saveAsTextFile(DATA + "/ca_ville")`, lister le dossier (`os.listdir`), relire avec `sc.textFile`. Que contient le fichier `_SUCCESS` ?

Reponses 5.3

[73]:
```
ca_ville.coalesce(1).saveAsTextFile(DATA + "/ca_ville")
print(os.listdir(DATA + "/ca_ville"))
print(sc.textFile(DATA + "/ca_ville").collect())
```

```
['._SUCCESS.crc', '_SUCCESS', 'part-00000', '.part-00000.crc']
["('Casablanca', 34000.0)", "('Fes', 35050.0)", "('Rabat', 42100.0)", "('Marrake
ch', 48600.0)", "('Agadir', 14000.0)"]
```

Partie 6 : Analyses métier

Répondre à chaque question par **un seul pipeline** (une chaîne de transformations terminée par une action).

Q6.1 Quel est le **produit le plus vendu** en quantité totale ?

reponse 6.1

[76]:
```
print(ventes.map(lambda v: (v[3], v[4]))
            .reduceByKey(lambda a, b: a + b)
            .max(key=lambda x: x[1]))
```

```
('Souris', 65)
```

Q6.2 Quelle ville a le **montant moyen par vente** le plus élevé ?

reponse 6.2

[78]:
```
print(ventes.map(lambda v: (v[2], (v[4] * v[5], 1)))
            .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
            .mapValues(lambda p: p[0] / p[1])
            .max(key=lambda x: x[1]))
```

```
('Marrakech', 16200.0)
```

Q6.3 Pour chaque produit, la **liste triée et sans doublon des villes** où il est vendu.

reponse 6.3

[80]:
```
print(ventes.map(lambda v: (v[3], v[2]))
            .distinct()
            .groupByKey()
            .mapValues(lambda vs: sorted(vs))
            .collect())
```

```
[('Tablette', ['Fes', 'Marrakech', 'Rabat']), ('Clavier', ['Agadir', 'Casablanca', 'Fes']),
('Laptop', ['Agadir', 'Casablanca', 'Fes', 'Marrakech', 'Rabat']), ('Telephone', ['Casablanc
a', 'Marrakech', 'Rabat']), ('Souris', ['Casablanca', 'Fes', 'Rabat'])]
```

Q6.4 Quelles ventes (identifiants) ont un montant **supérieur à la moyenne globale** ? (Calculer d'abord la moyenne avec une action, puis filtrer.)

reponse 6.4

```
2]: montants = ventes.map(lambda v: v[4] * v[5])
    moyenne = montants.mean()          # action : 9144.74
    print(ventes.filter(lambda v: v[4] * v[5] > moyenne)
                .map(lambda v: v[0])
                .collect())
```

```
[1, 2, 3, 4, 5, 6, 8, 9, 18, 21]
```

Q6.5 Quel est le **chiffre d'affaires par mois** (clé = les 7 premiers caractères de la date) ?

reponse 6.5

```
4]: print(ventes.map(lambda v: (v[1][:7], v[4] * v[5]))
                .reduceByKey(lambda a, b: a + b)
                .collect())
```

```
[('2025-01', 129200.0), ('2025-02', 44550.0)]
```